

Documentación Técnica

Stack tecnológico

Capa	Tecnología	Librerías principales
Frontend	Next.js (App Router)	React 19, Tailwind CSS
Backend	Express.js + Node.js	TypeORM, JWT, Bcrypt
Base de datos	PostgreSQL	Manejada via TypeORM
Testing	Jest	ts-jest

Arquitectura general

El sistema sigue una arquitectura de tres capas desacopladas que se comunican a través de una REST API:

- **Capa de presentación:** Aplicación Next.js que sirve la interfaz para feriantes y administradores.
- **Capa de lógica:** Backend Express.js organizado con el patrón Controller → Service → Repository.
- **Capa de datos:** Base de datos PostgreSQL accedida mediante TypeORM.

Estructura del backend

El backend está organizado en tres subcapas con responsabilidades bien definidas:

- **/api** — Rutas y controladores. Reciben las peticiones HTTP, validan el formato y delegan al servicio correspondiente.
- **/domain** — Núcleo de la aplicación. Contiene los servicios con la lógica de negocio y los modelos de dominio.
- **/infra** — Repositorios (acceso a la BD con TypeORM) y Mappers (transformación entre entidades de BD y objetos de dominio).

Rutas disponibles

Prefijo	Descripción
/auth	Registro, login, logout, refresh de token
/feria	CRUD de ferias, inscripción, listados
/usuario	Perfil y actualización de datos del usuario

Autenticación y seguridad

La autenticación se implementa con **JWT** (JSON Web Tokens):

- Al hacer login, el servidor emite un **access token** (expira en 15 minutos) y un **refresh token** (expira en 7 días).
- El refresh token se almacena en una cookie `httpOnly` para evitar acceso desde JavaScript.
- Las rutas protegidas validan el token mediante middleware (`authMiddleware` para usuarios, `adminMiddleware` para administradores).
- Las contraseñas se hashean con **bcrypt** antes de persistirse.

Modelo de datos

El sistema tiene tres entidades principales:

- Usuario** — almacena nombre, email, contraseña hasheada, rol (`admin` / `user`), rubro, teléfono y el estado de su última inscripción (`ultimaInscripcion`).
- Feria** — almacena nombre, fecha, horarios, dirección, cupo máximo y el flag `listasGeneradas` que indica si ya se generaron las listas de prioridad.
- Inscripcion** — tabla intermedia con clave compuesta (`usuarioId`, `feriaId`), estado y timestamp de inscripción.

Lógica de inscripciones y criterios de selección

Este es el módulo central del sistema. Cuando hay más inscriptos que cupo disponible, se aplican estrategias de ordenamiento implementadas con el **patrón Strategy**.

Estados posibles de un feriante

Estado	Descripción
pendiente	Nunca participó en una feria
confirmado	Participó y fue aprobado
suplente	Quedó en lista de espera
rechazado	No fue seleccionado (inhabilitado para la siguiente feria)
proximo	Cumplió la inhabilitación, puede volver a participar

Estrategias de ordenamiento (Criterio)

La clase abstracta `Criterio` define el método `generarListados` , que divide a los inscriptos en: **aprobados, lista de espera, rechazados y próximos**. Las implementaciones concretas son:

- **OrdenLlegada** — ordena estrictamente por fecha de inscripción (`createdAt`).
- **PrioridadInicial** — agrupa por estado en este orden: `pendiente` → `suplente` → `confirmado` → `rechazado` . Dentro de cada grupo, se ordena por llegada.
- **PrioridadRegenerada** — se usa cuando las listas ya se generaron y llegan nuevos inscriptos. Los usuarios `pendiente` se aprueban automáticamente (superando el cupo), y el orden entre `confirmado` y `suplente` se invierte respecto a la generación inicial.

Caducidad de rechazados (`reglasInscripcion`)

Al generar las listas de prioridad por primera vez en una feria, el sistema actualiza automáticamente los estados:

`rechazado` → `proximo` (cumplió la penalización)
`proximo` → `suplente` (puede volver a participar con menor prioridad)

Variables de entorno requeridas

```
DB_HOST=  
DB_PORT=5432  
DB_USER=  
DB_PASS=  
DB_NAME=  
JWT_SECRET=  
JWT_REFRESH_SECRET=  
PORT=4000  
NODE_ENV=development
```